

Java Collections Differences Interview Questions and Answers

By Ramesh Fadatare (Java Guides)

(Key points from Udemy Course: [Java Developer Interview Mastery: The Only Course You Need](#))

Difference Between List and Set in Java

Aspect	List	Set
Ordering	Maintains insertion order. Elements appear in the order they were added.	No guaranteed order. <code>HashSet</code> may appear random. Use <code>LinkedHashSet</code> or <code>TreeSet</code> for ordering.
Duplicates	Allows duplicates. Same value can appear multiple times.	Does not allow duplicates. Repeated values are ignored.
Null Elements	Allows null values. <code>ArrayList</code> allows multiple nulls.	Varies by implementation. <code>HashSet</code> allows one null. <code>TreeSet</code> may throw an exception.
Access & Implementations	Common types: <code>ArrayList</code> , <code>LinkedList</code> . Allows index-based access (<code>get(index)</code>).	Common types: <code>HashSet</code> , <code>LinkedHashSet</code> , <code>TreeSet</code> . No index access — focuses on value lookup.
Performance	Fast for indexed access (like <code>get(3)</code>). Performance may slow with many duplicate checks.	Optimized for uniqueness. Fast for <code>add</code> , <code>remove</code> , and <code>contains</code> in large collections.
Use Case	Best for ordered data with possible duplicates: logs, inputs, to-do lists.	Best for unique, non-repeating data: tags, IDs, usernames.

Java Collection vs Collections

Aspect	Collection	Collections
Type	An interface — defines structure but no implementation	A utility class — provides helper methods to work with collections
Purpose	Represents a group of elements — like a <code>List</code> , <code>Set</code> , or <code>Queue</code>	Offers static utility methods — like <code>sort()</code> , <code>reverse()</code> , <code>shuffle()</code> , <code>synchronizedList()</code>
Thread Safety	Implementations are not thread-safe by default	Provides methods to wrap collections for thread safety , like <code>synchronizedList()</code>
Method Types	Since Java 8, includes static and default methods	Contains only static methods — can be called without creating an instance

Java HashSet vs LinkedHashSet

Aspect	HashSet	LinkedHashSet
Underlying Data Structure	Uses a Hash Table only	Uses a Hash Table + Linked List to preserve insertion order
Ordering of Elements	No guaranteed order — element sequence may vary	Maintains the order of insertion
Performance	Faster for insertions, deletions, and lookups	Slightly slower due to maintaining insertion order
Null Handling	Allows one null element	Also allows one null element
Memory Overhead	Lower memory usage — only stores elements and hash buckets	Higher memory usage — stores links to previous and next elements
Iteration	Faster iteration — no ordering to maintain	Slower iteration — follows linked list to maintain order
Removal of Elements	Fast removal via hash lookup	Removes element and preserves order of remaining elements
Use Case	Use when order doesn't matter — e.g. tags, lookup sets	Use when order matters — e.g. recent items, user commands, UI menus

HashMap vs LinkedHashMap in Java

Aspect	HashMap	LinkedHashMap 
Ordering of Elements	✗ No order guarantee – iteration order is unpredictable	✓ Maintains insertion order (or access order , if enabled)
Internal Data Structure	Uses a Hash Table only	Combines Hash Table + Linked List to preserve order
Performance	✓ Faster for basic operations (<code>put</code> , <code>get</code> , <code>remove</code>)	⚠ Slightly slower due to managing the linked list
Memory Usage	✓ Lower memory usage – stores only keys, values, and hash buckets	✗ Higher memory usage – stores extra links (prev/next) per entry
Use Cases	Best for unordered, fast lookups – configs, cache, key-value mappings	Best for order-sensitive tasks – recent activity logs, UI lists
Iteration Order	✗ Unpredictable iteration order	✓ Predictable iteration (insertion or access order)
Null Handling	Allows one null key , multiple null values	Same: one null key , unlimited null values
Inheritance	Implements <code>Map</code> interface directly	Extends <code>HashMap</code> , adds ordering logic

Java HashMap vs ConcurrentHashMap

Aspect	HashMap	ConcurrentHashMap
Thread Safety	❌ Not thread-safe — no internal synchronization	✅ Thread-safe — designed for safe concurrent access
Java Version	Introduced in JDK 1.2	Introduced in JDK 1.5, part of <code>java.util.concurrent</code>
Null Handling	✅ Allows 1 null key, multiple null values	❌ Disallows null keys and values — avoids ambiguity during concurrent access
Iteration Behavior	❌ Fail-fast iterator — throws <code>ConcurrentModificationException</code>	✅ Fail-safe iterator — works on a snapshot, no exceptions during concurrent modifications
Performance	✅ Faster in single-threaded scenarios	⚠️ Slightly slower due to locking, but optimized for multithreading with lock striping
Use Case	Best for single-threaded applications — config maps, local lookups	Best for multi-threaded environments — caches, sessions, real-time metrics
Design Goal	Simplicity and speed	Scalability and concurrency safety

Java HashSet vs LinkedHashSet vs TreeSet

Aspect	HashSet	LinkedHashSet	TreeSet
Underlying Structure	🔗 Uses a Hash Table	🔗 Hash Table + Linked List	🌳 Uses a Red-Black Tree (sorted tree)
Ordering	❌ No order guarantee	✅ Maintains insertion order	✅ Maintains sorted order (natural or custom via Comparator)
Performance	⚡ Fastest — constant-time operations	⚡ Slightly slower due to maintaining order	🐢 Slowest — operations take $O(\log n)$ due to tree balancing
Null Handling	✅ Allows 1 null	✅ Allows 1 null	❌ Does not allow null — throws <code>NullPointerException</code>
Memory Usage	📦 Lowest memory — stores only hash data	📦 Medium — stores hash data + pointers for linked list	📦 Highest — stores full tree structure (left, right, parent refs)
Iteration Speed	🚀 Fast, but unpredictable order	🚀 Fast with predictable (insertion) order	🐢 Slower, but maintains sorted iteration
Use Cases	🔍 Best when you just need uniqueness + speed (e.g., deduping values)	📄 Use when you need uniqueness + order (e.g., recent logs, UI menus)	📊 Use when you need sorted unique data (e.g., rankings, autocomplete)

Fail-Fast vs Fail-Safe Iterators in Java

Aspect	Fail-Fast Iterators	Fail-Safe Iterators
Modification Handling	❌ Does not allow modification of a collection during iteration.	✅ Allows modification of a collection during iteration.
Exception Behavior	❗ Throws <code>ConcurrentModificationException</code> if the collection is modified while iterating.	✅ No exception thrown — safe to modify while iterating.
How It Works	🔍 Uses the original collection to traverse elements.	📄 Uses a copy of the original collection to traverse elements.
Memory Usage	💾 No extra memory needed — no duplication involved.	🧠 Requires extra memory to clone the collection.
Examples	📄 Iterators returned by <code>ArrayList</code> , <code>Vector</code> , <code>HashMap</code> .	📄 Iterator returned by <code>ConcurrentHashMap</code> .

CopyOnWriteArraySet vs HashSet in Java

Aspect	HashSet	CopyOnWriteArraySet
Thread Safety	❌ Not thread-safe. Requires external synchronization in multi-threaded scenarios.	✅ Thread-safe by design. Creates a fresh copy on each write.
Performance	⚡ High performance for add, remove, and lookup in single-threaded use.	🐢 Slower writes due to copying array; ✅ Fast, lock-free reads.
Backing Data Structure	📦 Backed by a <code>HashMap</code> .	📦 Backed by a <code>CopyOnWriteArrayList</code> .
Use Case	🔧 Ideal for non-concurrent apps, caches, sets with frequent updates.	👥 Best for read-heavy, concurrent apps — like listeners or feature flags.
Null Handling	✅ Allows one null element.	✅ Also allows null values — handle with care in concurrent logic.
Iterator Behavior	❗ Fail-Fast. Throws <code>ConcurrentModificationException</code> on unsynchronized concurrent updates.	✅ Fail-Safe. Iterates over a snapshot; no exceptions during concurrent modifications.

Difference Between Comparable and Comparator in Java

Aspect	Comparable	Comparator
Purpose	Defines the natural/default sorting order of a class.	Defines external/custom sorting rules outside the class.
Where It Is Used	The class itself decides how objects should be sorted.	A separate object decides how two objects are compared.
Sorting Type	Provides one natural sorting logic.	Can provide multiple different sorting logics.
Method	compareTo(Object o) — compares current object with another object.	compare(Object o1, Object o2) — compares two external objects.
Package	java.lang	java.util
Modifies Source Class?	Yes — requires modifying the actual class to implement Comparable.	No — keeps the original class unchanged.
Flexibility	Not flexible — only one sorting order is possible.	Very flexible — unlimited sorting strategies can be created.
Used When?	When objects must always follow a single natural order.	When you need multiple sorting orders or when you cannot modify the original class.

Sorting Mechanism	Collections.sort() and Arrays.sort() use compareTo if the class implements Comparable.	If class does <i>not</i> implement Comparable, sorting must be done using a Comparator.
Rule Memory Trick	"Object compares itself with another object."	"A third object compares two other objects."